Zero Trust Access Policy with Okta Adaptive MFA

Design and Implementation of Contextual Access Controls — documented by Rithik Kumar

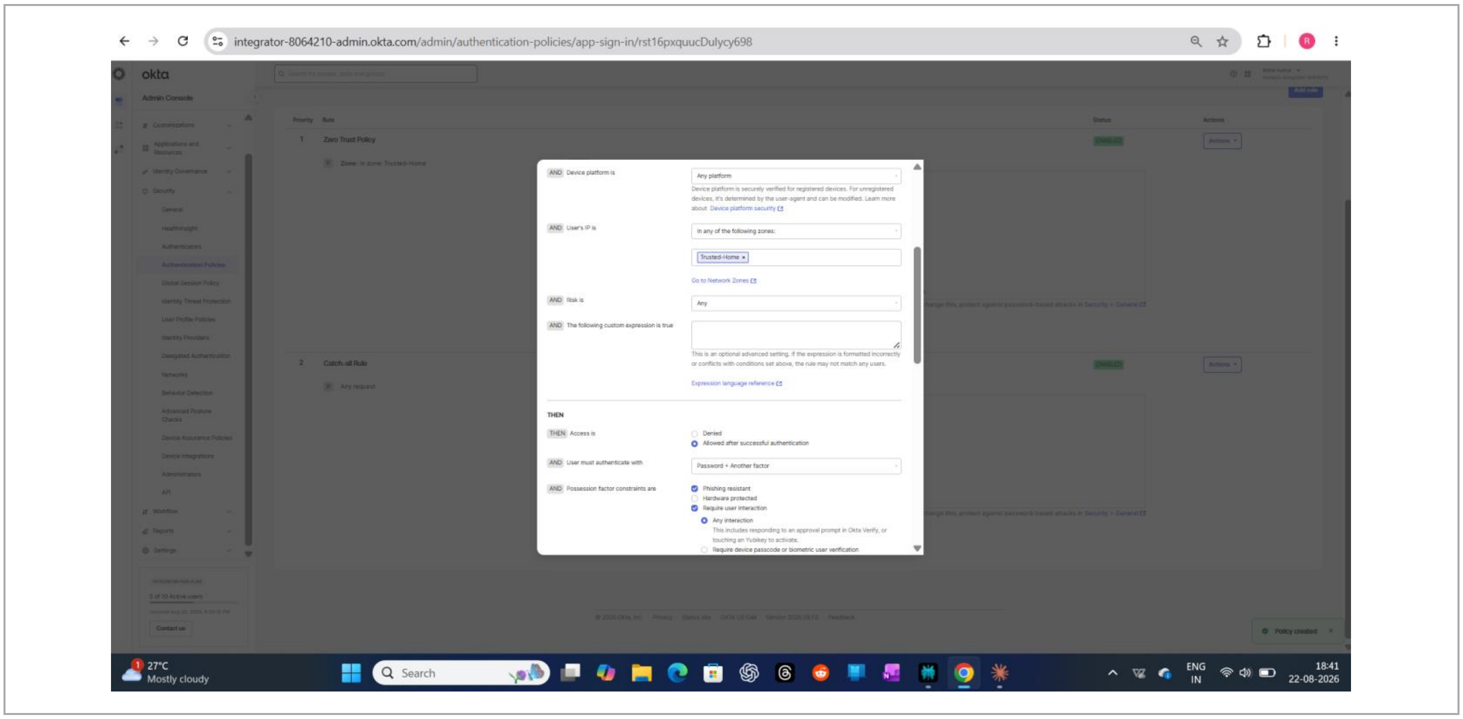
This project builds directly on the network zones and identity foundation from my earlier Okta work: a genuine adaptive authentication policy where the SAME login gets treated differently depending on context — which network it's coming from, what the device looks like, and what Okta's own risk signals say about it.

I'm including the parts of this that didn't go smoothly on purpose. I hit two separate errors while testing — a sign-in failure and a 403 Forbidden — and rather than just re-trying until it worked, I pulled the raw System Log data into a spreadsheet to actually find out what was happening underneath the UI. That debugging process is honestly the part of this project

PHASE 1 — DESIGNING THE ADAPTIVE POLICY

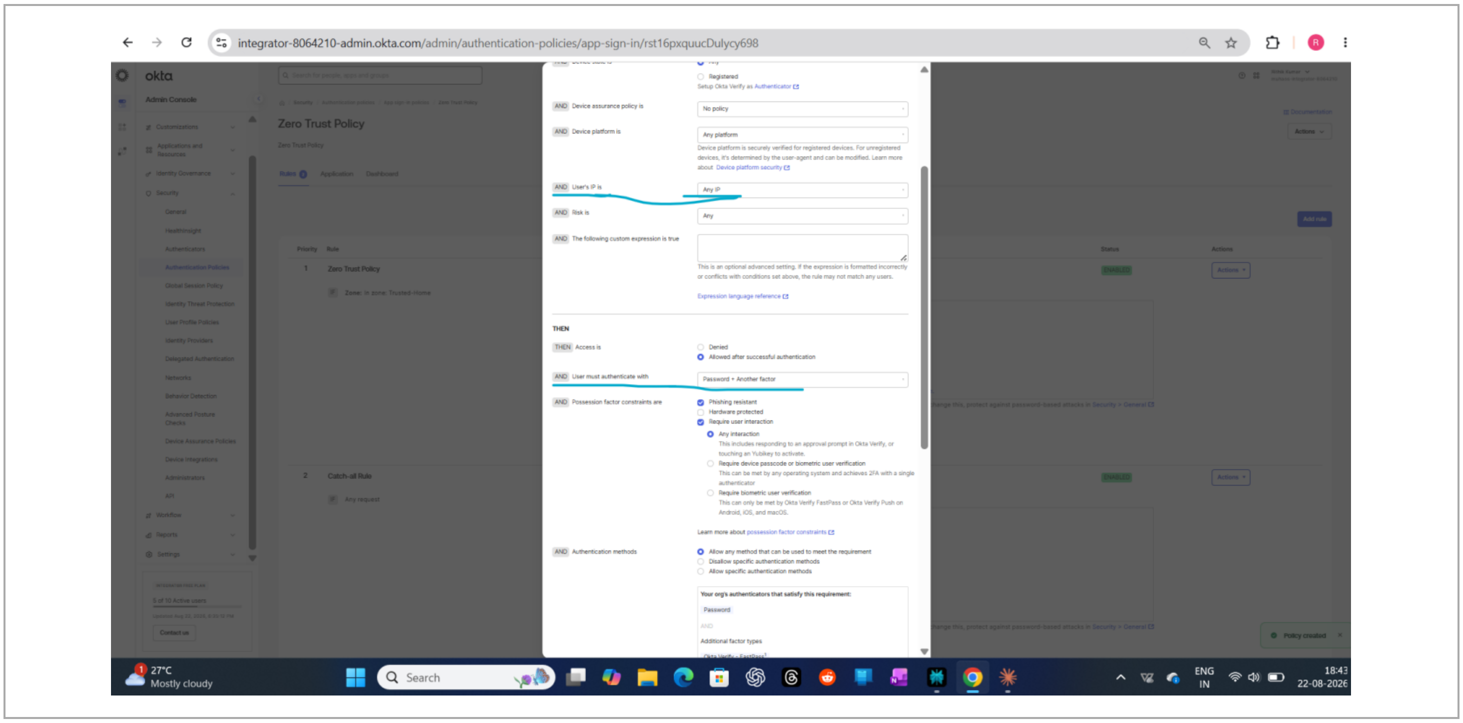
Step 1 — Building the Rule Logic

Started by defining the actual conditions inside the policy rule — network zone, device platform, and risk level — and what should be required when they match: password plus another factor, with phishing-resistant and user-interaction constraints turned on.



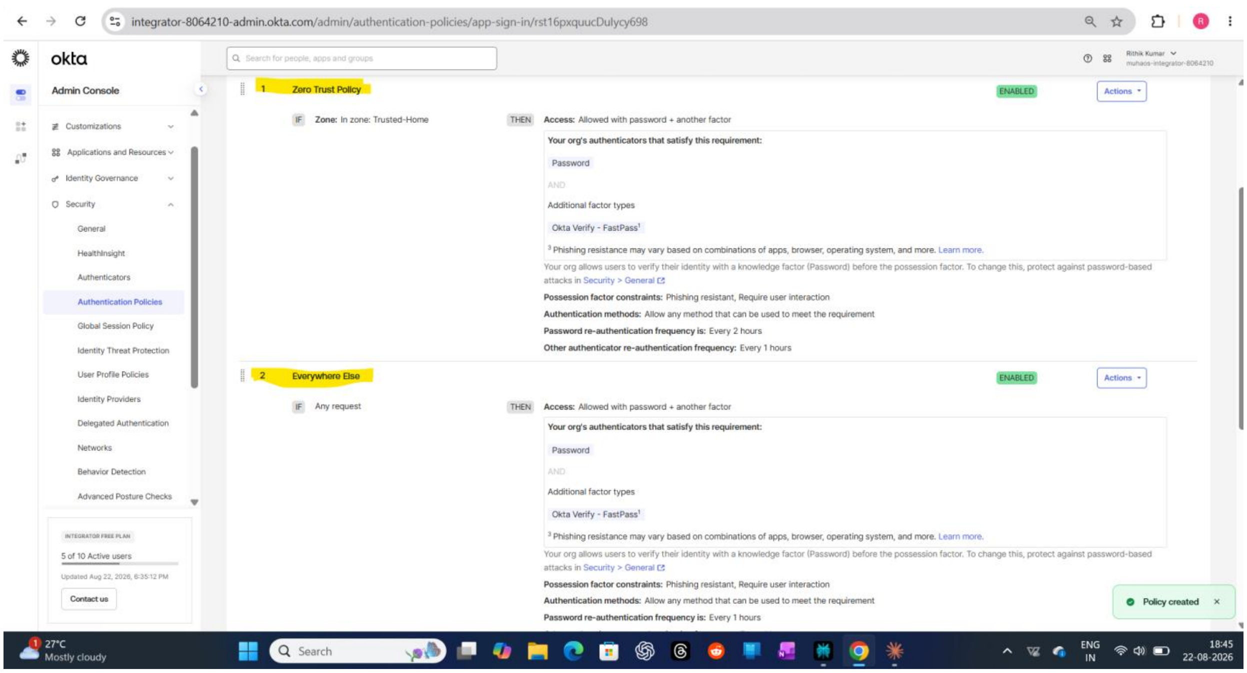
Step 2 — Reviewing the Full Condition Set

Scrolled through the rest of the rule to double check the IP/zone condition and the authentication requirement together — wanted to actually understand every field before saving, not just click through it.



Step 3 — Both Rules, Side by Side

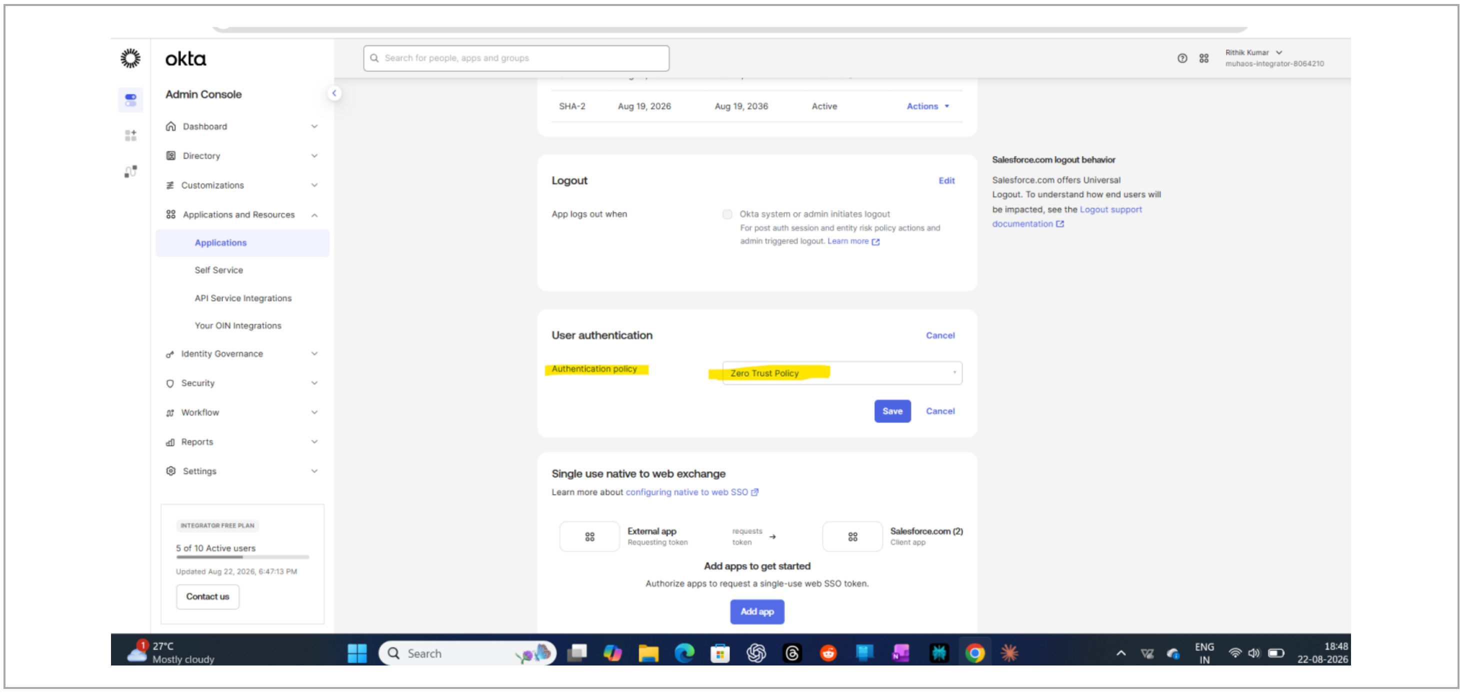
Final policy: Rule 1 matches the Trusted-Home zone, Rule 2 is the catch-all for everywhere else. I ended up requiring password + another factor on both rules rather than relaxing it for the trusted zone — in hindsight, a stricter baseline felt like the more defensible real-world choice than optimizing for convenience on my own home network.



PHASE 2 — APPLYING IT TO A REAL APP

Step 4 — Attaching the Policy to Salesforce

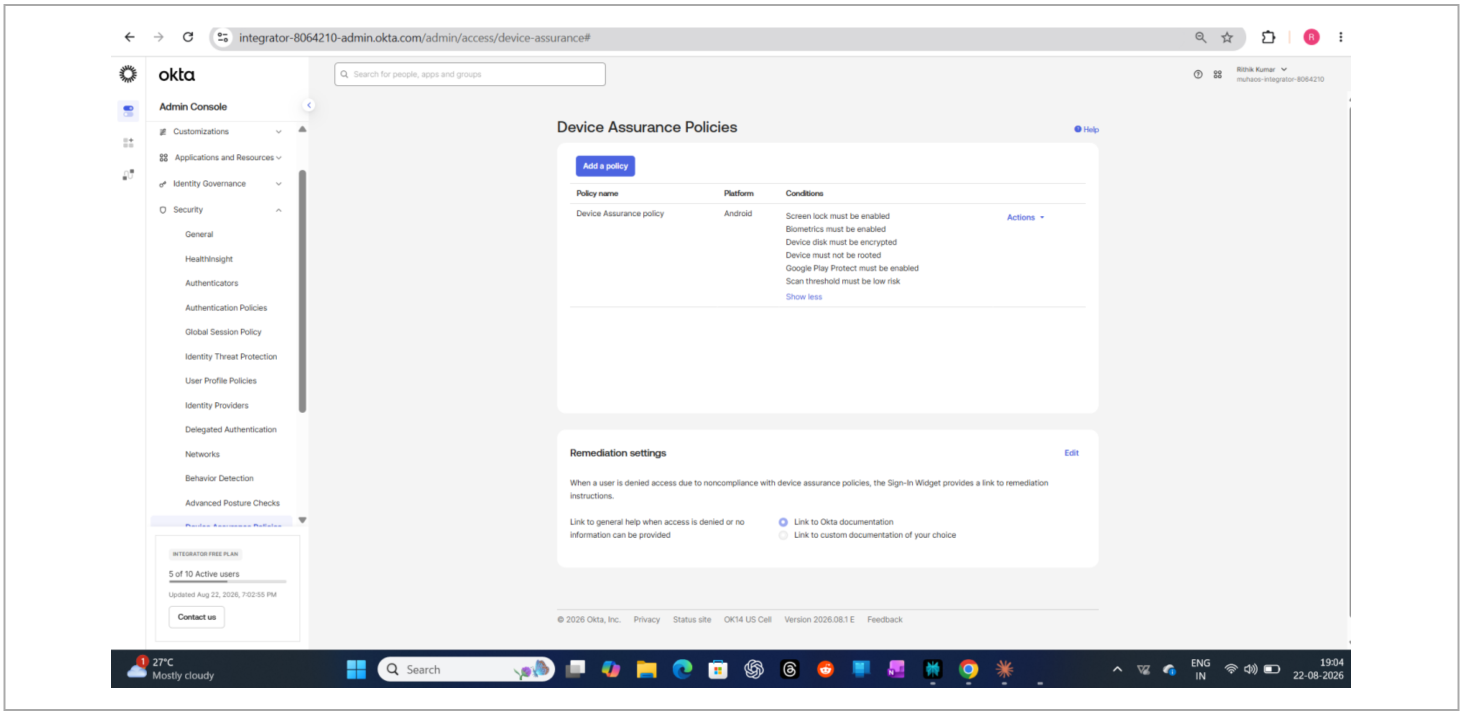
Switched the Salesforce app's authentication policy from Default to my new Zero Trust Policy — this is the point where the policy actually starts controlling real sign-ins, not just sitting there configured.



PHASE 3 — A BONUS FIND

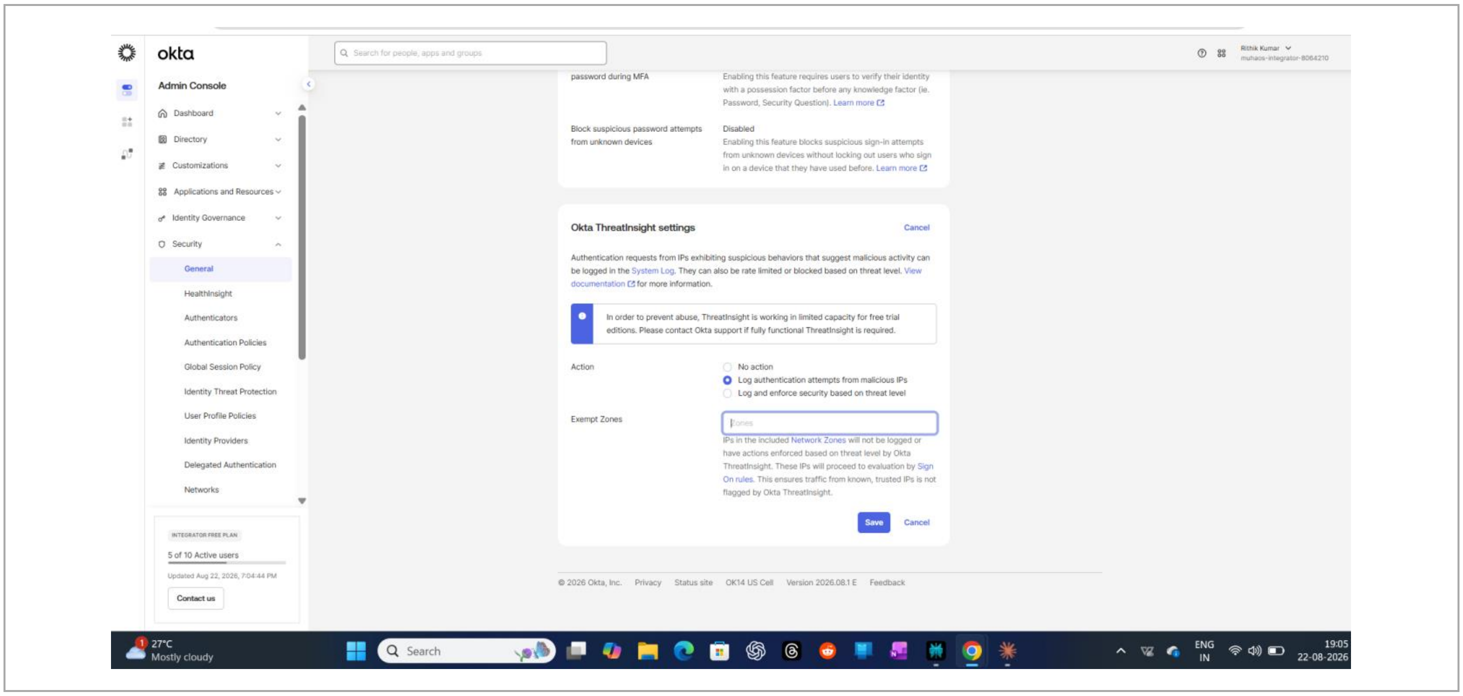
Step 5 — Device Assurance Was Actually Available

Wasn't sure this would be available on the free tier, but it was — an existing Device Assurance policy checking screen lock, biometrics, disk encryption, root status, and Google Play Protect. Good reminder to just go check rather than assume a feature is locked behind a paid plan.



Step 6 — Turning On ThreatInsight

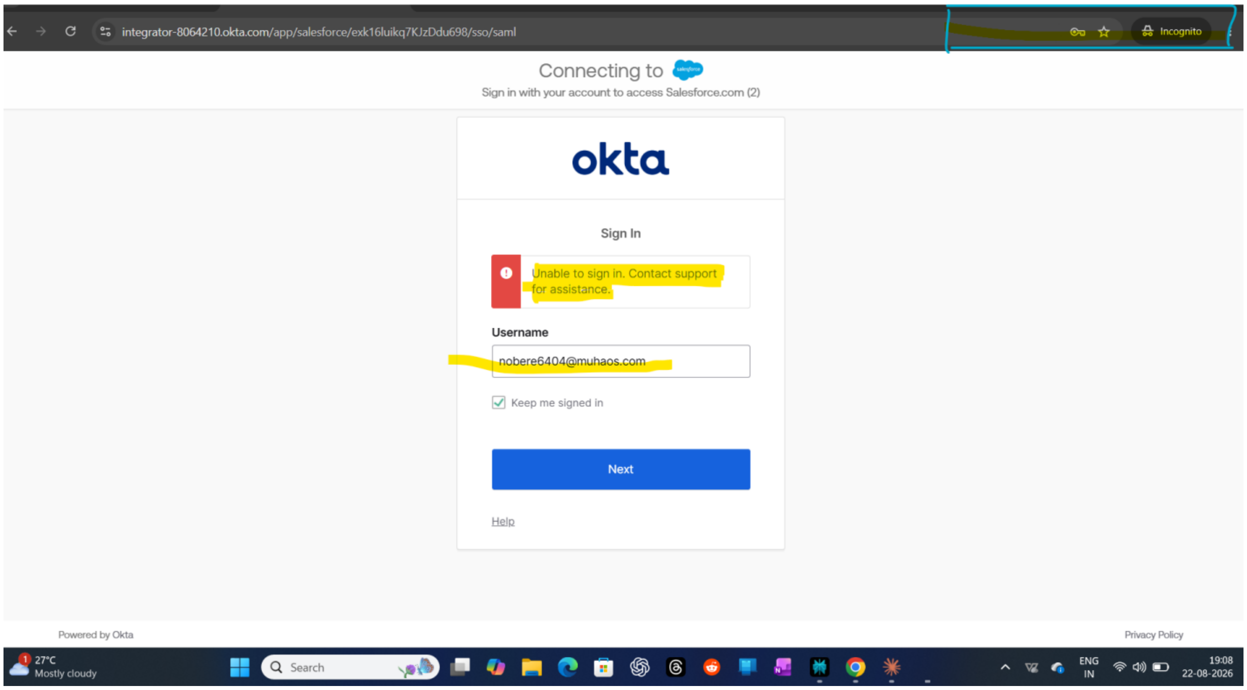
Enabled Okta ThreatInsight to log authentication attempts from suspicious IPs. It runs in limited capacity on the free tier, but it's still a real, functioning risk-signal layer on top of the zone-based rules.



PHASE 4 — TESTING IT (AND HITTING REAL PROBLEMS)

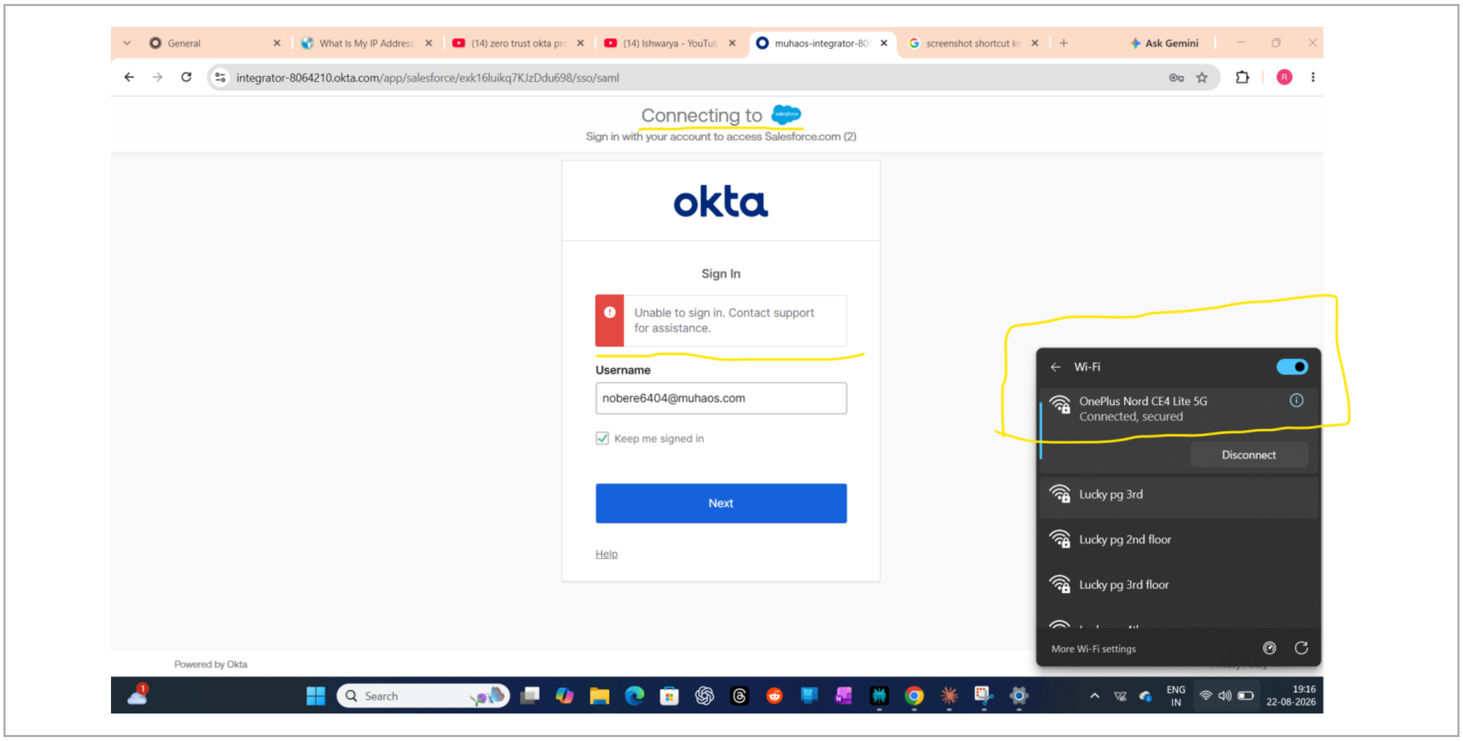
Step 7 — First Real Test — An Error, Not a Clean Success

Tried signing in through an incognito window to test the policy cleanly. Got "Unable to sign in. Contact support for assistance." instead of the smooth pass/fail I expected. Honestly, my first reaction was to assume I'd broken the policy config.



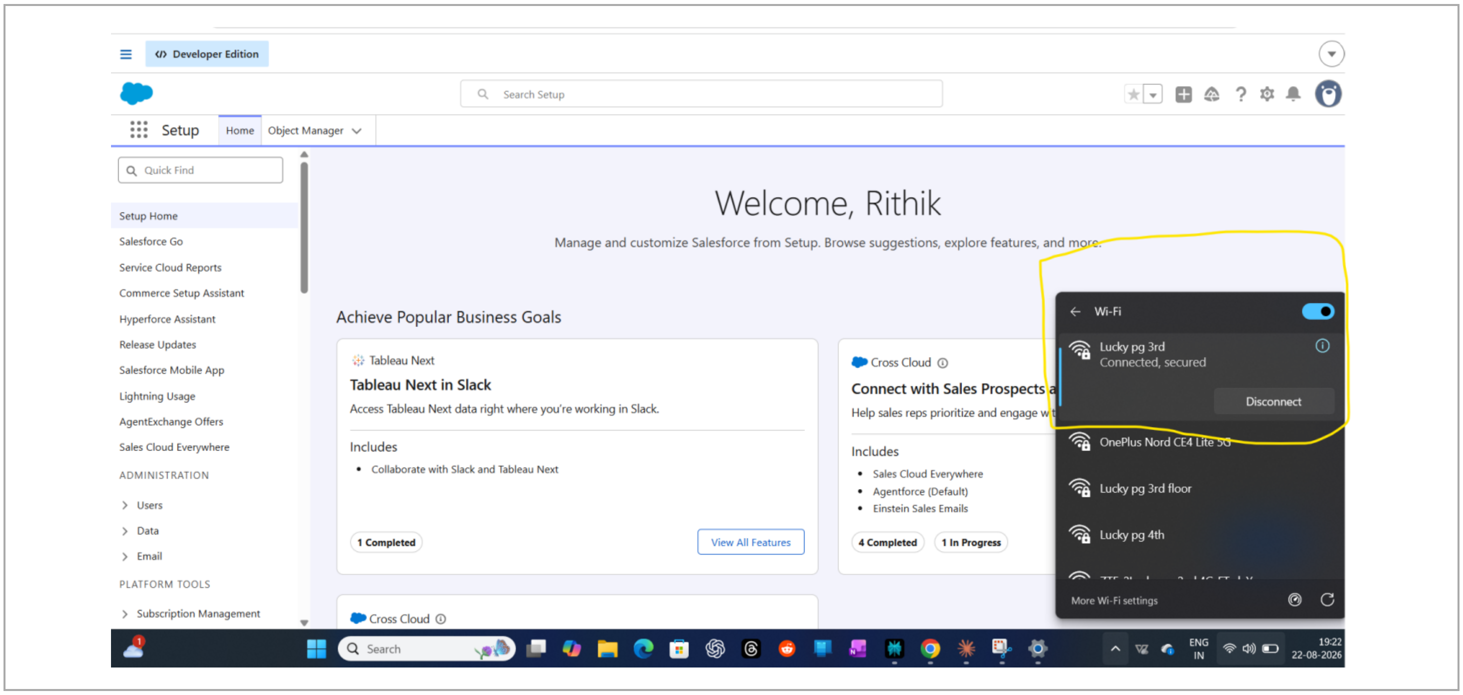
Step 8 — Same Error, Different Network

Switched to mobile network to properly test the "untrusted network" branch of the policy — same sign-in error came back. At this point I wasn't sure if it was the policy, the app config, or something else entirely.



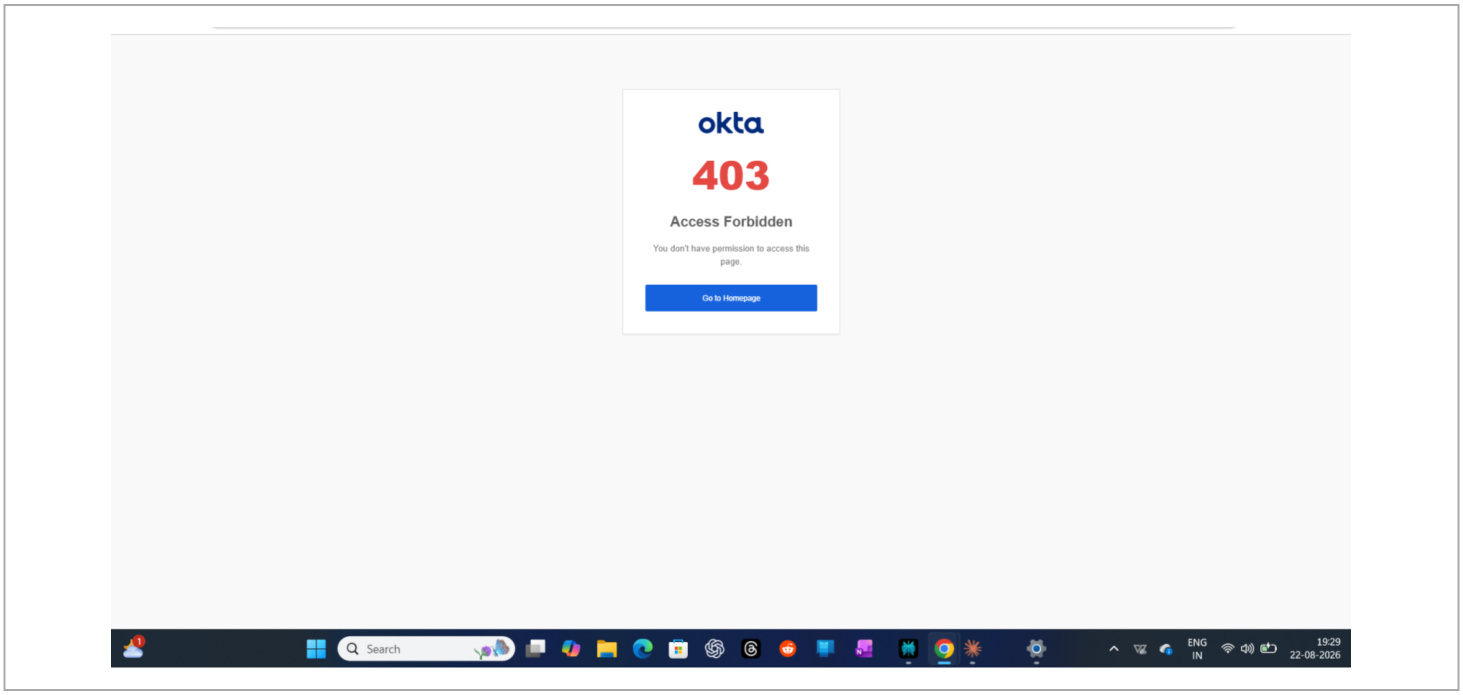
Step 9 — Ruling Out the Network Itself

Cycled through a few different wifi networks to make sure the error wasn't just a flaky connection issue on my end before assuming it was a config problem on Okta's side.



Step 10 — A Second, Different Error Shows Up

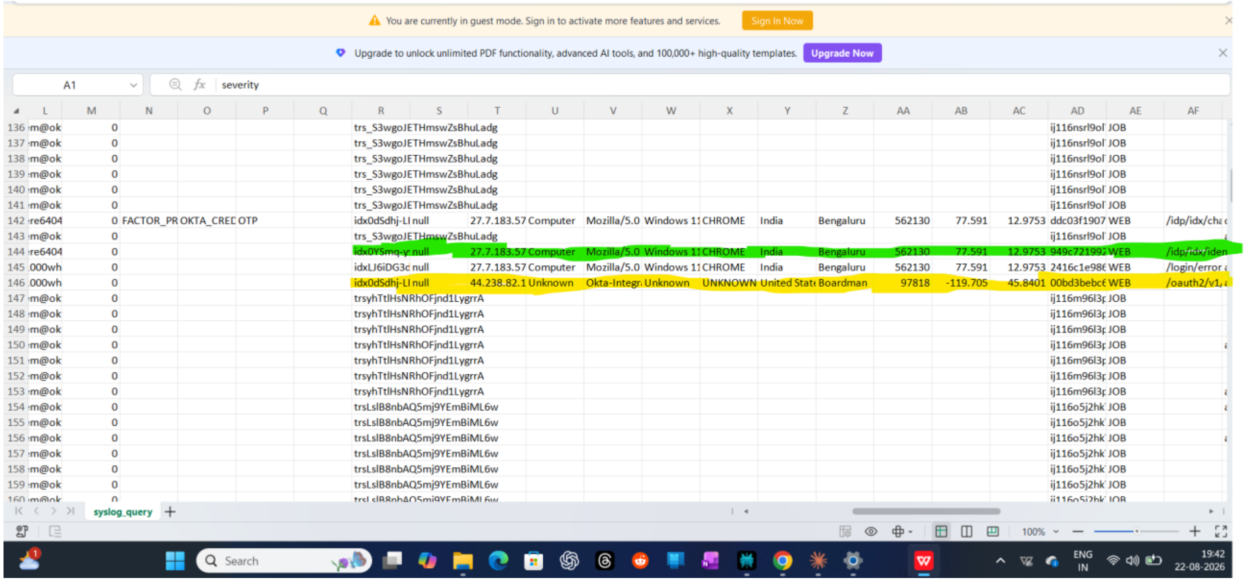
At one point hit a completely different error — a 403 Access Forbidden on the Okta side itself. Two different failure modes in one testing session was honestly a bit frustrating, but it meant there were two separate things to actually go investigate instead of one.



PHASE 5 — DEBUGGING WITH REAL DATA INSTEAD OF GUESSING

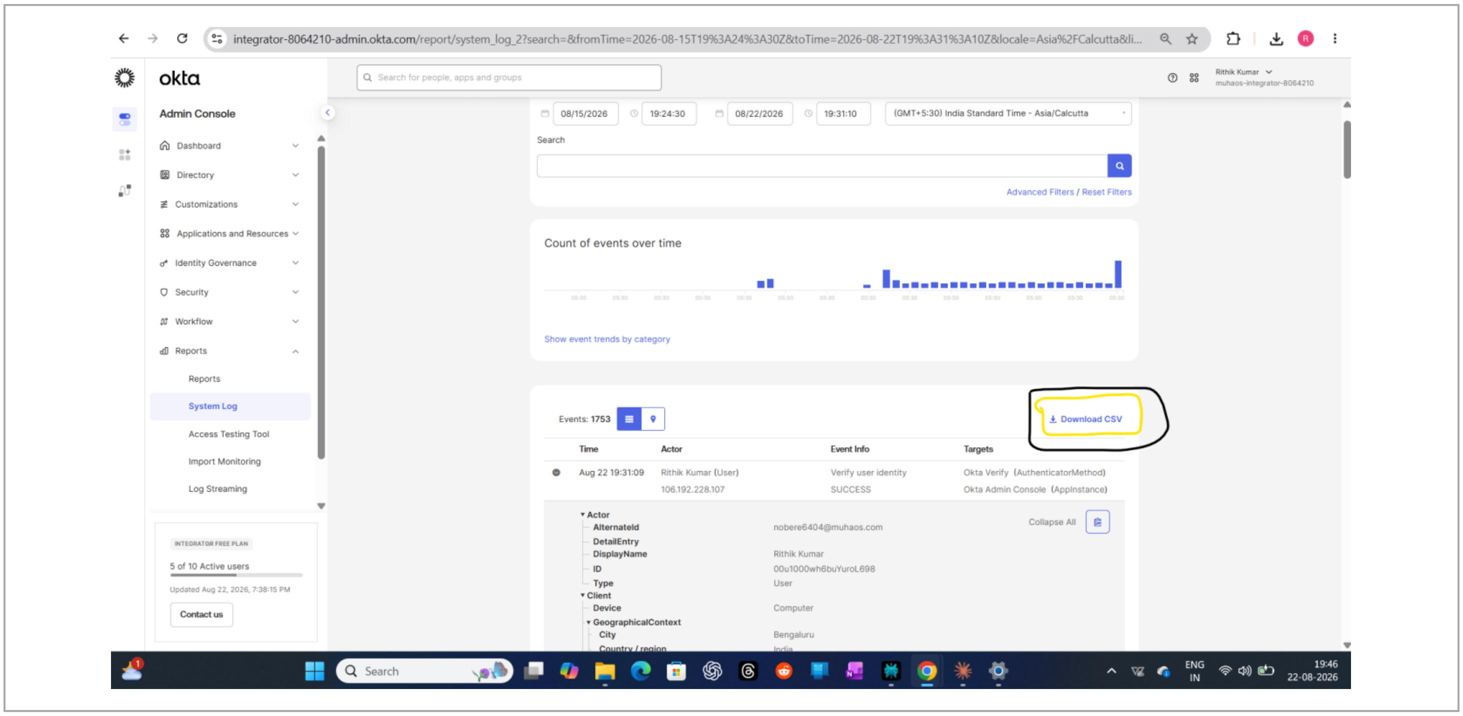
Step 11 — Pulling the Raw System Log Into a Spreadsheet

Rather than keep guessing, I downloaded the System Log as a CSV and opened it in a spreadsheet to actually read the raw event data — factor types, IP addresses, event outcomes, row by row. This is where I found the actual detail behind what was failing, instead of just the summary Okta shows in the UI.



Step 12 — Confirmed: The Policy Genuinely Works

Went back into the System Log UI and found it — a successful "Verify user identity" event through Okta Verify, tied to my account, from my actual location. 1,753 events logged across all my testing. The adaptive MFA challenge was firing correctly; the earlier errors turned out to be session/browser-state issues from switching networks mid-session, not a broken policy.



A Personal Note on This Project (and the Three Before It)

This is the third hands-on project I've built this month, and I want to be honest about something: before this, my IAM knowledge was mostly certifications, study material, and watching other people configure things in videos. Reading about SAML, MFA policies, and Zero Trust concepts is one kind of understanding. Actually sitting with a broken sign-in error at 7 PM, not knowing if it's my policy, my network, or the app config — and working through it step by step until the System Log finally showed me the truth — is a completely different kind of understanding.

If you're hiring for an IAM Engineer role, or know someone who is — I'd love to talk.
